Module 24: Window Functions (Most Important Advanced SQL Topic)

> Window Functions perform calculations across a set of rows related to the current row without collapsing the result set. They are one of the most powerful SQL features for analytics, reporting, dashboards, and interviews.

📖 Introduction

Consider an Employees table:

EmployeeID	EmployeeName	Salary
1	Kunal	50000
2	Rahul	70000
3	Aman	60000

Management asks:

> Rank employees based on salary.

Expected Output:

EmployeeName	Salary	Rank
Rahul	70000	1
Aman	60000	2
Kunal	50000	3

Window Functions solve this efficiently.

🎯 Learning Objectives

After completing this module, you will be able to:

✅ Understand Window Functions

✅ Use OVER()

✅ Use ROW_NUMBER()

✅ Use RANK()

✅ Use DENSE_RANK()

✅ Use LEAD() and LAG()

✅ Calculate Running Totals

✅ Calculate Moving Averages

✅ Solve Advanced Analytics Problems

📚 Table of Contents

1️⃣ What are Window Functions?

Window Functions perform calculations across a group of rows while preserving individual rows.

Normal Aggregate

```sql
SELECT AVG(Salary)
FROM Employees;
```

Output:

```
Single Row
```

Window Function

```sql
SELECT EmployeeName,
       Salary,
       AVG(Salary) OVER()
FROM Employees;
```

Output:

EmployeeName	Salary	AvgSalary
Kunal	50000	60000
Rahul	70000	60000
Aman	60000	60000

Rows remain visible.

Why Window Functions Matter

Used in:

- KPI Dashboards
- Financial Analysis
- Sales Analytics
- Ranking Reports
- Data Science

2️⃣ Why Window Functions are Needed

Without Window Functions:

```
GROUP BY
```

collapses rows.

With Window Functions:

```
Detailed Rows
+
Aggregated Insights
```

Example:

```
Employee Salary
Employee Rank
Department Average
```

all in same query.

3️⃣ OVER() Clause

The heart of Window Functions.

Syntax

```
Function()
OVER
(
    PARTITION BY ...
    ORDER BY ...
)
```

Example

```
SELECT EmployeeName,
       Salary,
       AVG(Salary) OVER()
FROM Employees;
```

OVER defines the window.

Components

```
PARTITION BY
ORDER BY
```

4️⃣ ROW_NUMBER()

Assigns unique row numbers.

Example

```
SELECT EmployeeName,
       Salary,

       ROW_NUMBER()
       OVER
       (
           ORDER BY Salary DESC
       ) AS RowNum

FROM Employees;
```

Output

EmployeeName	Salary	RowNum
Rahul	70000	1
Aman	60000	2
Kunal	50000	3

Uses

- Pagination
- Top-N Analysis
- Duplicate Removal

Example: Remove Duplicates

```
WITH DuplicateRows AS
(
    SELECT *,
           ROW_NUMBER()
           OVER
           (
               PARTITION BY Email
               ORDER BY CustomerID
           ) AS RN
    FROM Customers
)

DELETE
FROM DuplicateRows
WHERE RN > 1;
```

5️⃣ RANK()

Assigns ranks with gaps.

Example

Employee	Salary
Rahul	70000
Aman	70000
Kunal	50000

Query

```
SELECT EmployeeName,
       Salary,

       RANK()
       OVER
       (
           ORDER BY Salary DESC
       ) AS RankNo

FROM Employees;
```

Output

Employee	Salary	Rank
Rahul	70000	1
Aman	70000	1
Kunal	50000	3

Notice:

```
Rank 2 Missing
```

6 DENSE_RANK()

Assigns ranks without gaps.

Query

```sql
SELECT EmployeeName,
       Salary,

       DENSE_RANK()
       OVER
       (
           ORDER BY Salary DESC
       ) AS DenseRank

FROM Employees;
```

Output

Employee	Salary	Rank
Rahul	70000	1
Aman	70000	1
Kunal	50000	2

Difference

Function	Gaps
RANK()	Yes
DENSE_RANK()	No

7️⃣ NTILE()

Divides rows into groups.

Example

```
SELECT EmployeeName,
       Salary,

       NTILE(4)
       OVER
       (
           ORDER BY Salary DESC
       ) AS Quartile

FROM Employees;
```

Uses

- Quartiles
- Percentiles
- Customer Segmentation

Example

```
Top 25%
Middle 25%
Bottom 25%
```

8️⃣ LEAD()

Access next row.

Example

```
SELECT EmployeeName,
       Salary,

       LEAD(Salary)
       OVER
       (
           ORDER BY Salary
       ) AS NextSalary

FROM Employees;
```

Output

Salary	NextSalary
50000	60000
60000	70000
70000	NULL

Uses

- Trend Analysis
- Forecasting
- Comparing periods

9️⃣ LAG()

Access previous row.

Query

```
SELECT EmployeeName,
       Salary,

       LAG(Salary)
       OVER
       (
           ORDER BY Salary
       ) AS PreviousSalary

FROM Employees;
```

Output

Salary	PreviousSalary
50000	NULL
60000	50000
70000	60000

Uses

- Month-over-Month Growth
- Year-over-Year Analysis

🔟 FIRST_VALUE()

Returns first value in window.

Query

```
SELECT EmployeeName,
       Salary,

       FIRST_VALUE(Salary)
       OVER
       (
           ORDER BY Salary DESC
       ) AS HighestSalary

FROM Employees;
```

Output

Highest salary repeated for all rows.

Uses

Finding top performer.

1️⃣1️⃣ LAST_VALUE()

Returns last value in window.

Query

```
SELECT EmployeeName,
       Salary,

       LAST_VALUE(Salary)
       OVER
       (
           ORDER BY Salary
           ROWS BETWEEN
           UNBOUNDED PRECEDING
           AND UNBOUNDED FOLLOWING
       ) AS HighestSalary

FROM Employees;
```

Used less frequently but important.

1️⃣2️⃣ Running Totals

Extremely common analytics question.

Sales Table

Month	Sales
Jan	100
Feb	200
Mar	300

Query

```
SELECT Month,
       Sales,

       SUM(Sales)
       OVER
       (
```

```
            ORDER BY Month
        ) AS RunningTotal

FROM Sales;
```

Output

Month	Sales	RunningTotal
Jan	100	100
Feb	200	300
Mar	300	600

Business Uses

- Revenue Tracking
- Budget Monitoring
- KPI Dashboards

1️⃣3️⃣ Moving Averages

Important analytics technique.

Query

```
SELECT Month,
       Sales,

       AVG(Sales)
       OVER
       (
           ORDER BY Month

           ROWS BETWEEN
           2 PRECEDING
           AND CURRENT ROW
       ) AS MovingAverage

FROM Sales;
```

Uses

- Trend Analysis
- Forecasting
- Financial Reporting

1️⃣4️⃣ PARTITION BY

Creates separate windows.

Example

```
SELECT EmployeeName,
       DepartmentID,
       Salary,

       AVG(Salary)
       OVER
       (
           PARTITION BY DepartmentID
       ) AS DepartmentAverage

FROM Employees;
```

Output

Each department gets its own average.

Similar To

```
GROUP BY
```

but rows remain visible.

Example

```
AVG(Salary)
```

```
OVER(PARTITION BY DepartmentID)
```

1️⃣5️⃣ Common Mistakes

Missing ORDER BY

Bad:

```
ROW_NUMBER() OVER()
```

Result may be unpredictable.

Confusing RANK and DENSE_RANK

Remember:

```
RANK → gaps
DENSE_RANK → no gaps
```

Incorrect LAST_VALUE

Requires window frame.

Ignoring PARTITION BY

Can produce wrong results.

1️⃣6️⃣ Best Practices

Always Understand Business Requirement

Choose correct function.

Use PARTITION BY Carefully

Separate logical groups.

Always Specify ORDER BY

For ranking functions.

Use Meaningful Aliases

Good:

```
DepartmentAverage
RunningTotal
EmployeeRank
```

Test Large Datasets

Window functions can be expensive.

Professional Example

```
SELECT EmployeeName,
       DepartmentID,
       Salary,

       RANK()
       OVER
       (
           PARTITION BY DepartmentID
           ORDER BY Salary DESC
       ) AS DepartmentRank

FROM Employees;
```

1️⃣7️⃣ Common Interview Questions

What is a Window Function?

Performs calculations across rows while preserving row detail.

Difference Between GROUP BY and Window Functions?

GROUP BY collapses rows.

Window Functions preserve rows.

Difference Between ROW_NUMBER and RANK?

ROW_NUMBER always unique.

RANK allows ties.

Difference Between RANK and DENSE_RANK?

DENSE_RANK has no gaps.

What is PARTITION BY?

Creates separate windows.

What is Running Total?

Cumulative sum over ordered rows.

1️⃣8️⃣ Business Analytics Examples

Top 3 Employees

```
WITH RankedEmployees AS
(
    SELECT EmployeeName,
           Salary,

           ROW_NUMBER()
           OVER
           (
               ORDER BY Salary DESC
           ) AS RN

    FROM Employees
```

```
)

SELECT *
FROM RankedEmployees
WHERE RN <= 3;
```

Department Ranking

```
SELECT EmployeeName,
       DepartmentID,

       RANK()
       OVER
       (
           PARTITION BY DepartmentID
           ORDER BY Salary DESC
       ) AS RankNo

FROM Employees;
```

Monthly Revenue Running Total

```
SELECT Month,
       Revenue,

       SUM(Revenue)
       OVER
       (
           ORDER BY Month
       ) AS RunningRevenue

FROM Sales;
```

Customer Quartiles

```
SELECT CustomerID,

       NTILE(4)
       OVER
       (
```

```
            ORDER BY TotalPurchase DESC
        ) AS Quartile

FROM Customers;
```

Month-over-Month Growth

```
SELECT Month,
       Revenue,

       Revenue -
       LAG(Revenue)
       OVER
       (
           ORDER BY Month
       ) AS Growth

FROM Sales;
```

📝 Summary

In this module, you learned:

- ✅ OVER()
- ✅ ROW_NUMBER()
- ✅ RANK()
- ✅ DENSE_RANK()
- ✅ NTILE()
- ✅ LEAD()
- ✅ LAG()
- ✅ FIRST_VALUE()
- ✅ LAST_VALUE()
- ✅ Running Totals
- ✅ Moving Averages
- ✅ PARTITION BY

🧠 Practice Questions

Theory

1. What is a Window Function?
2. Why are Window Functions important?
3. What is OVER()?
4. Difference between ROW_NUMBER and RANK?
5. Difference between RANK and DENSE_RANK?
6. What is PARTITION BY?
7. What is LEAD()?
8. What is LAG()?
9. What is a Running Total?
10. What is a Moving Average?

Practical Exercises

Task 1

Rank employees by salary.

Task 2

Find top 5 highest-paid employees.

Task 3

Calculate department-wise salary rankings.

Task 4

Create running sales totals.

Task 5

Calculate month-over-month revenue growth.

Task 6

Divide customers into quartiles.

Challenge Project

Create:

```
Employees
Departments
Customers
Orders
Sales
Products
```

Build:

- Top Performers Report
- Department Rankings
- Running Revenue Dashboard
- Customer Segmentation
- Growth Analysis
- Moving Average Reports

using Window Functions.

🚀 Next Module

➞ Module 25: Advanced SQL Analytics

Topics Covered:

- Cohort Analysis
- Retention Analysis
- Funnel Analysis
- Pivoting Data
- Dynamic Reports
- Advanced KPI Calculations
- Real Interview Case Studies
- End-to-End Analytics Projects